

TASK 1

CODE Explanation:

1. Make a class REGION_TRACKER to store necessary information like:
 - a. page number
 - b. lru
 - c. number of offsets
 - d. count of every offset (offsets[])
 - e. demand of the offset
 - f. issued and useful count (for accuracy purpose. both are arrays)
 - g. last line accessed in that region (page)
2. Initialize an array tracker of size 32, 64 ,128 to track the number of pages as you like.
3. Make two maps one to track issued prefetches and other to track prefetches in cache.
The value in both the map stores a pair that gives the index of page number and offset in the tracker variable and key is the prefetched line and filled line.
4. The l2c_prefetcher_operate functions has all the functionality of the prefetcher.
5. Extract basic information like:
 - a. cache line address
 - b. page number
 - c. page base address
 - d. page offset
6. check if the request has already been prefetched? If yes, remove it from filled_prefetches map and increase the useful count of the offset in tracker[idx].useful_count[idx]
7. check if the requested page is already being tracked in our tracker array? If no, add it to the tracker array and return.
8. calculate the relative offset in rel_offset.
9. check if this new offset is being tracked or not? If no, track this new offset.
10. calculate the degree based on average demand of only those offsets for which min demand is 2.
11. score calculation:
 - a. Collect candidate offsets: Iterates through all observed offsets in the current page and filters out invalid ones .
 - b. Compute accuracy: Uses a Laplace-smoothed ratio of useful vs. issued prefetches, with exponential decay applied to penalize offsets that generated wasted prefetches.
 - c. Normalize demand and score offsets: A hybrid score is calculated. We saw that prioritizing accuracy is more beneficial.
 - d. Rank top-k offsets: Maintains a small sorted list (best_idx[], best_count[]) of up to 5 highest-scoring offsets, ordered by their hybrid score.

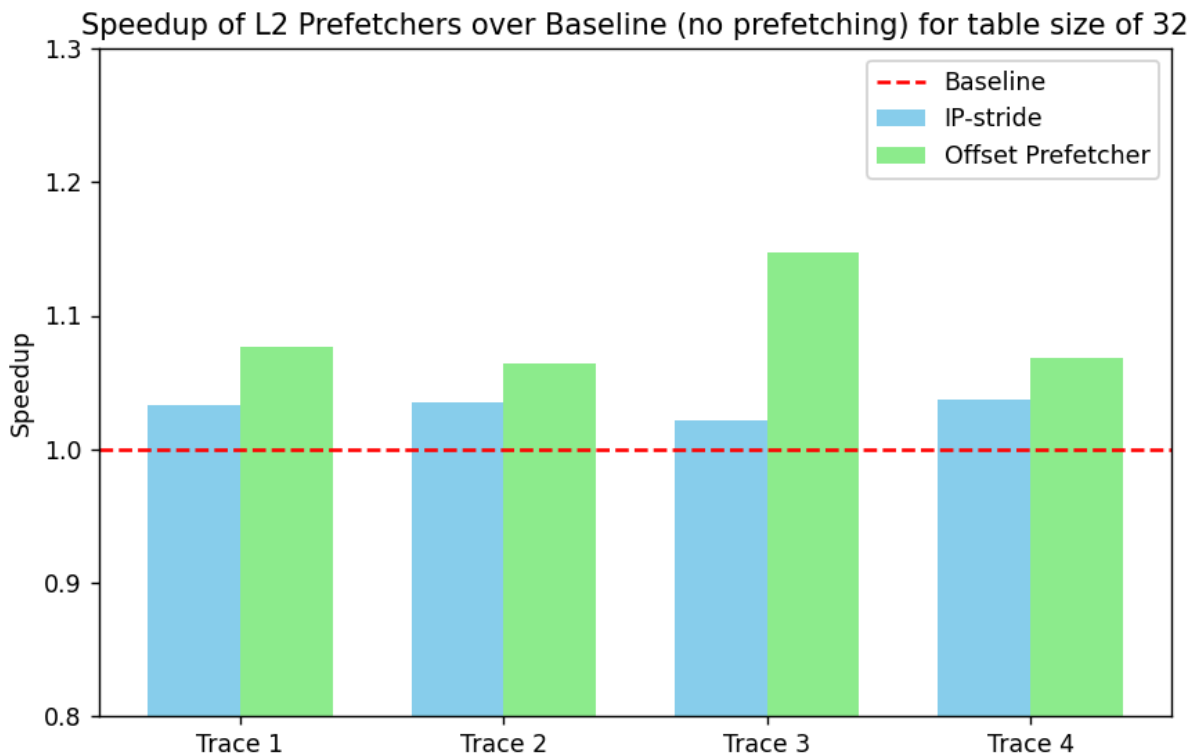
12. prefetching:

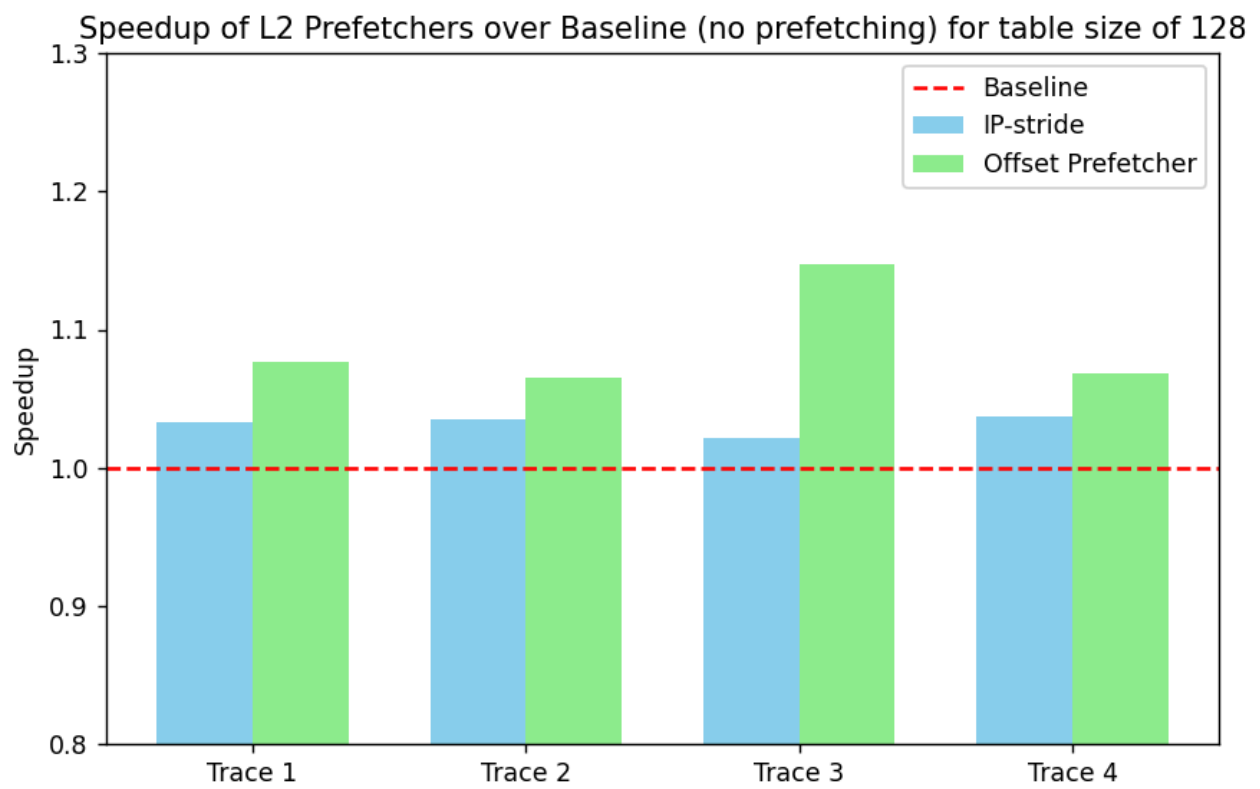
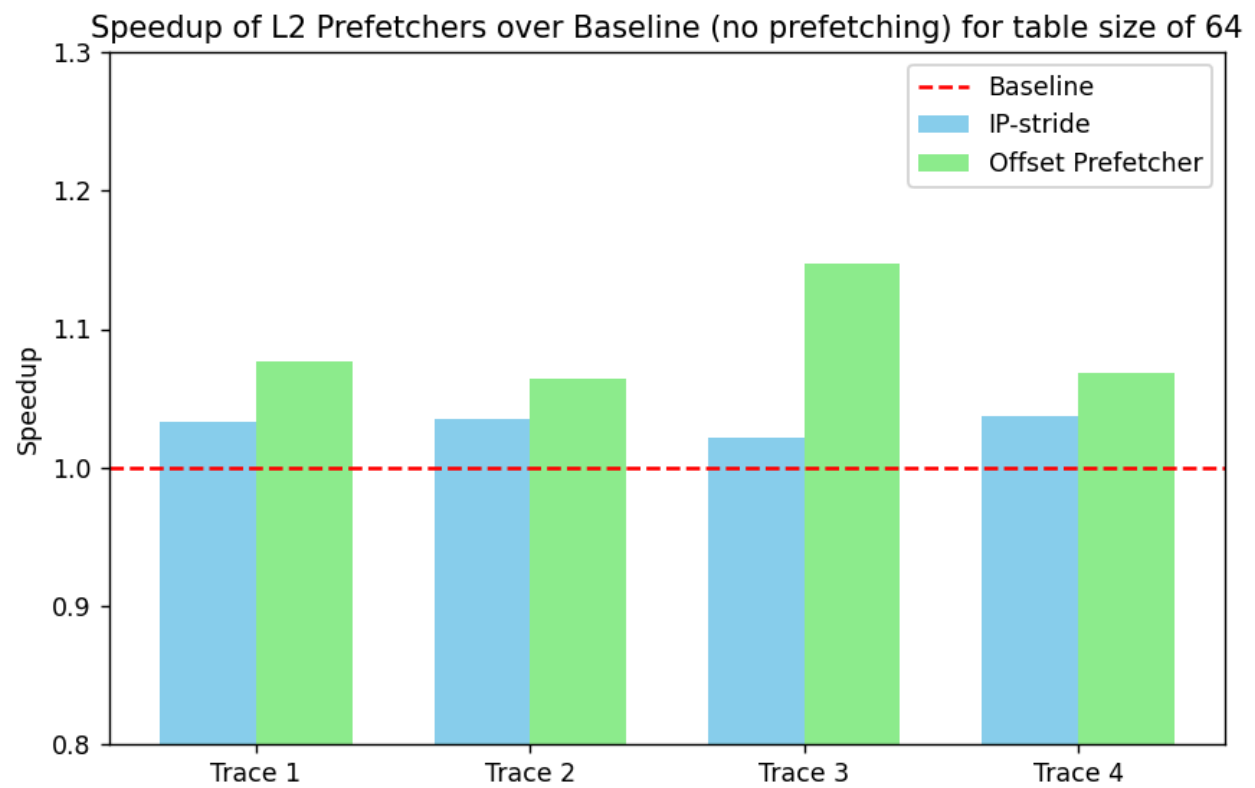
- Select and cap candidates: degree is capped to n best candidates.
- Check and form prefetch address: For each candidate offset, it validates the predicted line (within the page, not the current line) and computes its full memory address.
- Issue prefetch based on system pressure: If the MSHR is lightly loaded, prefetch goes to L2 cache; otherwise to LLC.

13. data coming to cache:

- Track filled prefetches: When a cache line is brought into L2 (cache_fill), it checks if this line corresponds to a previously issued prefetch (issued_origins).
- Mark successful prefetches: If the line was indeed a prefetch (prefetch == 1), the code records it in filled_prefetches, storing the origin info (which offset/pattern triggered it).
- Cleanup issued_origins : Once processed, the entry is removed from issued_origins to avoid double counting. If an evicted line was a filled prefetch, it is also removed from filled_prefetches.

PLOTS:





TABLES:

Baseline: Without prefetching non-inculsive			
TRACE	IPC	MPKI	Speed UP
trace1	0.529026	40.3453	1
trace2	0.462268	47.6893	1
trace3	0.834309	22.197	1
trace4	0.462276	47.562	1

IP Stride Prefetching			
TRACE	IPC	MPKI	Speed UP
trace1	0.54648	40.3407	1.032992707
trace2	0.478241	47.7267	1.034553549
trace3	0.852268	22.1965	1.021525598
trace4	0.47923	47.5704	1.03667506

Offset prefetching with table size 32			
TRACE	IPC	MPKI	Speed UP
trace1	0.569798	40.3337	1.077069936
trace2	0.491895	47.6992	1.064090528
trace3	0.957152	22.1972	1.147239212
trace4	0.493706	47.5665	1.067989686

Offset prefetching with table size 64			
TRACE	IPC	MPKI	Speed UP
trace1	0.569798	40.3337	1.077069936
trace2	0.491895	47.6992	1.064090528
trace3	0.957152	22.1972	1.147239212
trace4	0.493706	47.5665	1.067989686

Offset prefetching with table size 128			
TRACE	IPC	MPKI	Speed UP
trace1	0.569587	40.3453	1.07667109
trace2	0.492213	47.6965	1.06477844
trace3	0.957152	22.1972	1.147239212
trace4	0.493718	47.5708	1.068015644

TASK 2: Exclusive cache

Changes in PA2-CS683-ChampSim/inc/memory_class.h

Added a new memory class: `#define TRANSFER 8`

Changes in PA2-CS683-ChampSim/cache_hierarchies/exclusive_cache.cc

1. handle_fill()

- Changed the condition from adding an entry into WQ from just checking dirty bit, to the following:

```
((block[set][way].valid && ( cache_type == IS_L1D || cache_type == IS_L1I ||  
cache_type == IS_L2C)) || ( block[set][way].dirty && cache_type == IS_LLC ))
```

- Changed the writeback packet type based on the dirty bit:

```
if (block[set][way].dirty) {  
    writeback_packet.type = WRITEBACK;    // needs to be written  
back to lower level  
} else {  
    writeback_packet.type = TRANSFER;    // just moving ownership,  
not dirty  
}
```

- While fill into the cache from MSHR, invalidating the block in the other cache levels.

```
{  
    fill_cache(set, way, &MSHR.entry[mshr_index]);  
    const uint64_t block_addr = MSHR.entry[mshr_index].full_addr >>  
LOG2_BLOCK_SIZE;  
    if (cache_type == IS_L1D) {  
        int inval_result_l2 =  
ooo_cpu[cpu].L2C.invalidate_entry(block_addr);  
        int inval_result_llc = uncore.LLC.invalidate_entry(block_addr);  
  
    } else if (cache_type == IS_L2C) {  
        int inval_result_llc = uncore.LLC.invalidate_entry(block_addr);  
    }  
}
```

2. handle_writeback()

- To write back, we need to evict a block, and add it to the WQ of the next level if dirty (for non inclusive cache). Changed the condition from just dirty to the following:

```
(block[set][way].valid && cache_type == IS_L2C) || ( block[set][way].dirty &&
cache_type == IS_LLC ))
```

b. Changed the writeback packet type based on the dirty bit:

```
if (block[set][way].dirty) {
    writeback_packet.type = WRITEBACK;    // needs to be
written back to lower level
} else {
    writeback_packet.type = TRANSFER;    // just moving
ownership, not dirty
}
```

c. While filling the block in the cache from the WQ

i. Invalidating the block in other cache levels:

```
if (lower_level && cache_type == IS_L2C) {
    auto next_cache = dynamic_cast<CACHE*>(lower_level);
    if (next_cache) {
        int inval_result =
next_cache->invalidate_entry(WQ.entry[index].full_addr >> LOG2_BLOCK_SIZE);
    }
}
```

ii. changing the dirty bit based on the packet type:

```
if (WQ.entry[index].type == TRANSFER)
    block[set][way].dirty = 0;    // line is just being transferred,
not modified
else
    block[set][way].dirty = 1;
```

3. handle_read()

a. If cache hit from an read queue entry, the assert whether the entries are in any other cache:

i. For L1D:

```
if (ooo_cpu[cpu].L2C.check_hit(&RQ.entry[index]) != -1)
    assert(0);

if (uncore.LLC.check_hit(&RQ.entry[index]) != -1)
    assert(0);
```

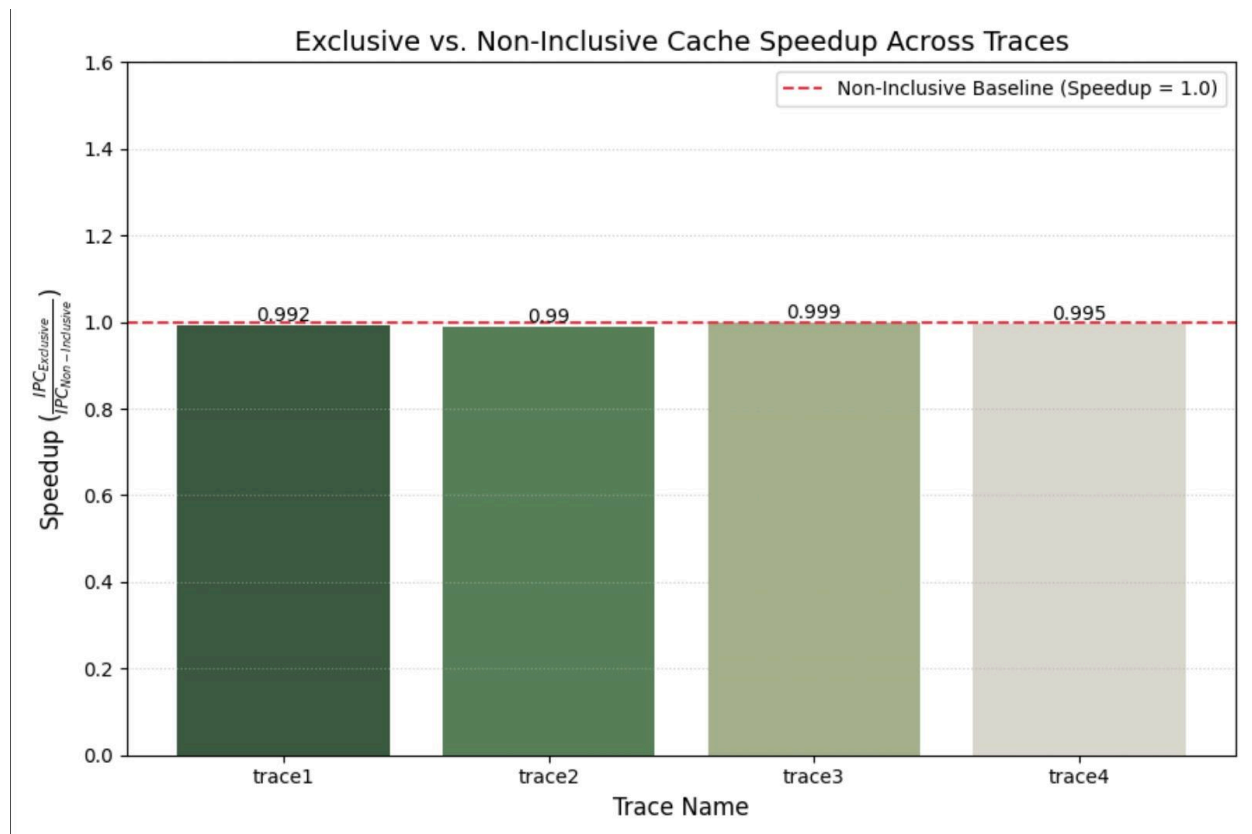
ii. For L2C:

```
if (uncore.LLC.check_hit(&RQ.entry[index]) != -1)
    assert(0);
```

The above changes are to ensure exclusivity property. So, invalidating the entry in lower levels of cache when filling and write back even if the block is not dirty are the main concerns.

The assert statements on a hit, ensured that the cache is indeed exclusive!.

Speedup of Exclusive over Non-Inclusive:



Trace	Configuration	IPC	L1D_MPKI	L2C_MPKI	LLC_MPKI	Speedup
trace1	Exclusive	0.5254	66.0505	96.4905	70.4327	0.9922
	Non-Inclusive	0.5295	66.0505	40.5813	40.1098	N/A
trace2	Exclusive	0.4581	78.022	113.838	83.1502	0.9899
	Non-Inclusive	0.4628	78.0219	47.742	47.3799	N/A
trace3	Exclusive	0.8383	35.7547	52.3442	38.2563	0.9989
	Non-Inclusive	0.8393	35.7547	22.0985	21.7126	N/A
trace4	Exclusive	0.4598	77.7246	113.466	82.8675	0.9952
	Non-Inclusive	0.4621	77.7237	47.6566	47.1964	N/A

Observations:

1. The reason for low IPC might be due to the STALL for adding into WQ. since every evicted line needs to be added to the WQ of the lower level, this might cause the lower IPC.
2. There is Higher MPKI because, most in exclusive cache, all the writebacks are misses. (exclusivity property; so hit % = 0).

TASK 3:

Over the changes made in Task 2, for converting non inclusive cache to exclusive cache, one change added to maintain exclusivity even when there is prefetching.

Changes in PA2-CS683-ChampSim/cache_hierarchies/exclusive_cache.cc

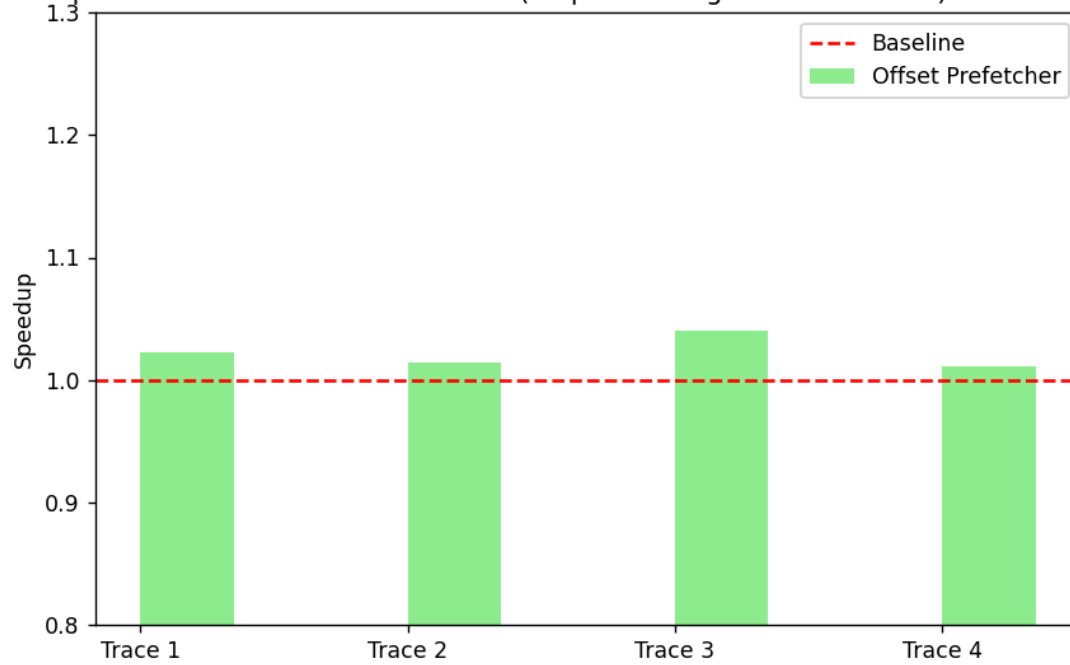
1. In add_pq():

```
        if((cache_type == IS_L2C && (ooo_cpu[cpu].L1D.check_hit(packet)!=-1)) ||  
(cache_type == IS_LLC && ((ooo_cpu[cpu].L1D.check_hit(packet)!=-1) ||  
(ooo_cpu[cpu].L2C.check_hit(packet)!=-1))))  
            return -1;
```

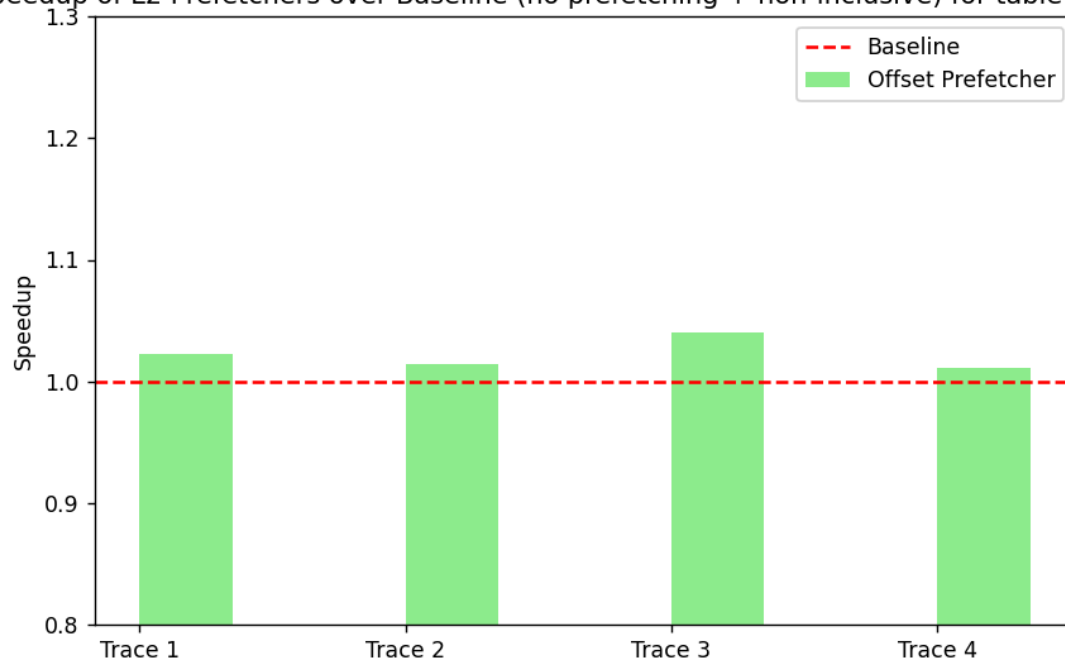
PLOTS:

non-inclusive:

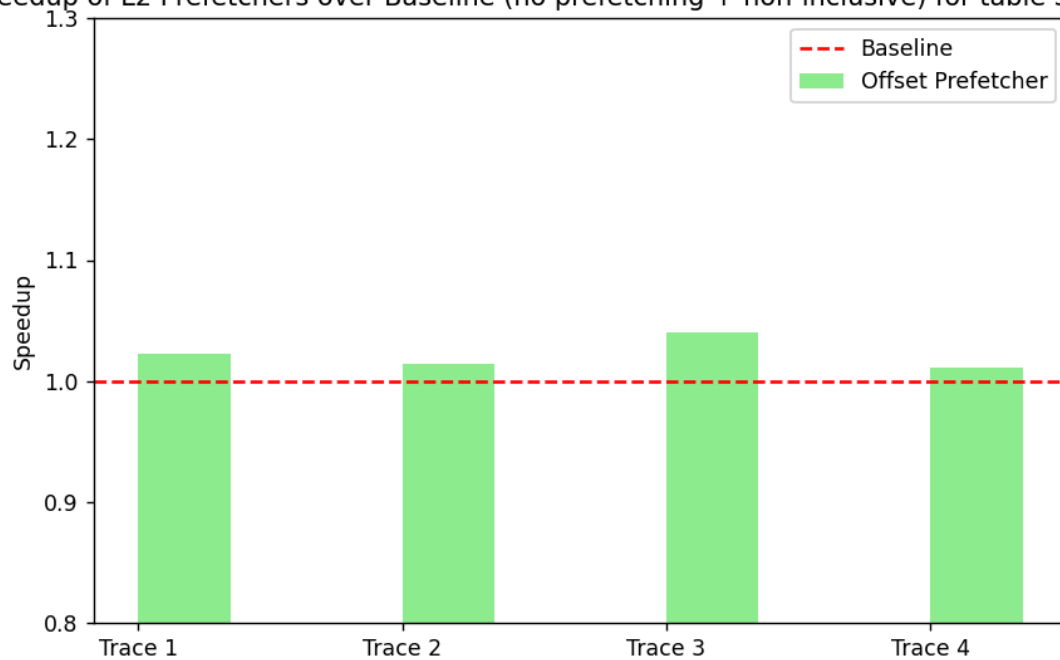
Speedup of L2 Prefetchers over Baseline (no prefetching + non-inclusive) for table size of 32



Speedup of L2 Prefetchers over Baseline (no prefetching + non-inclusive) for table size of 64

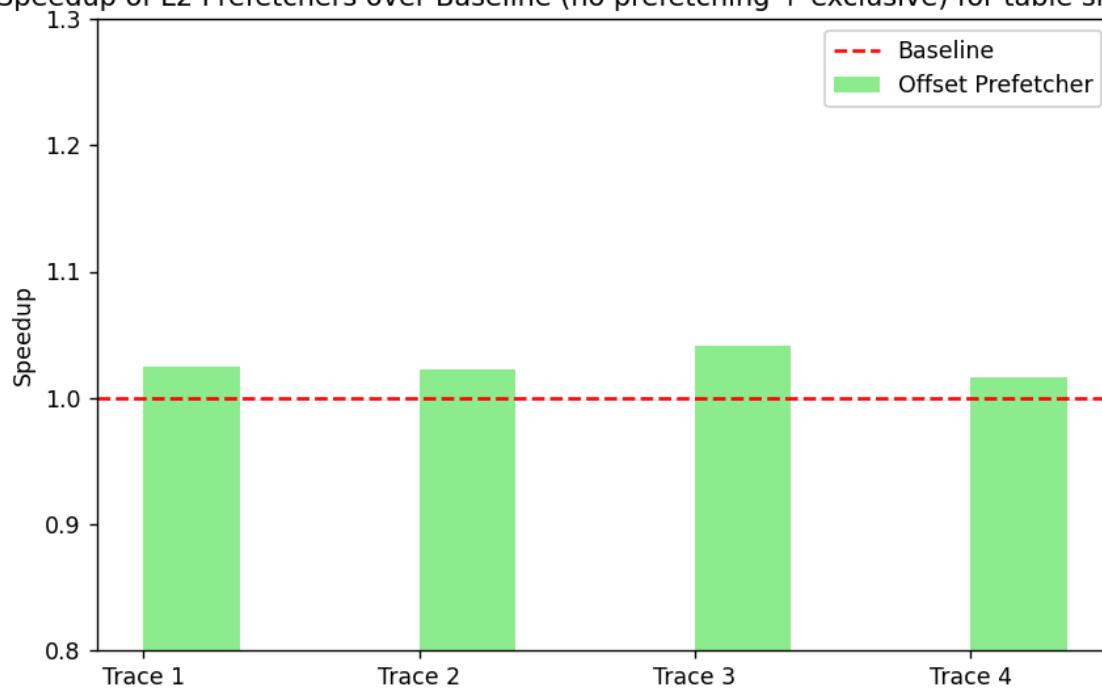


Speedup of L2 Prefetchers over Baseline (no prefetching + non-inclusive) for table size of 128

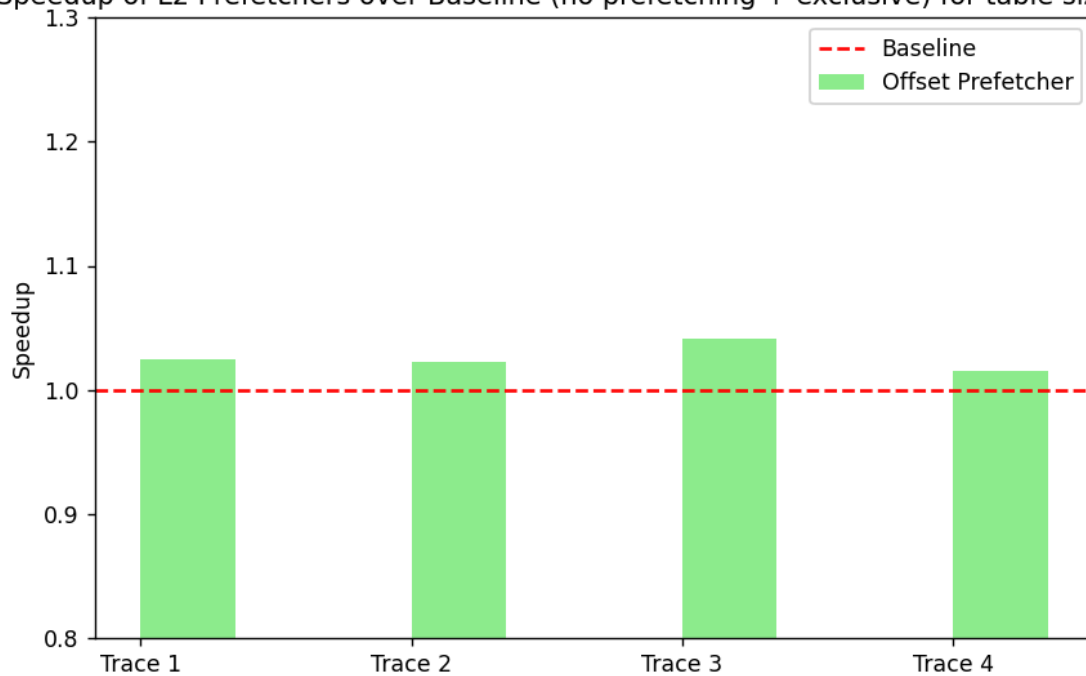


exclusive:

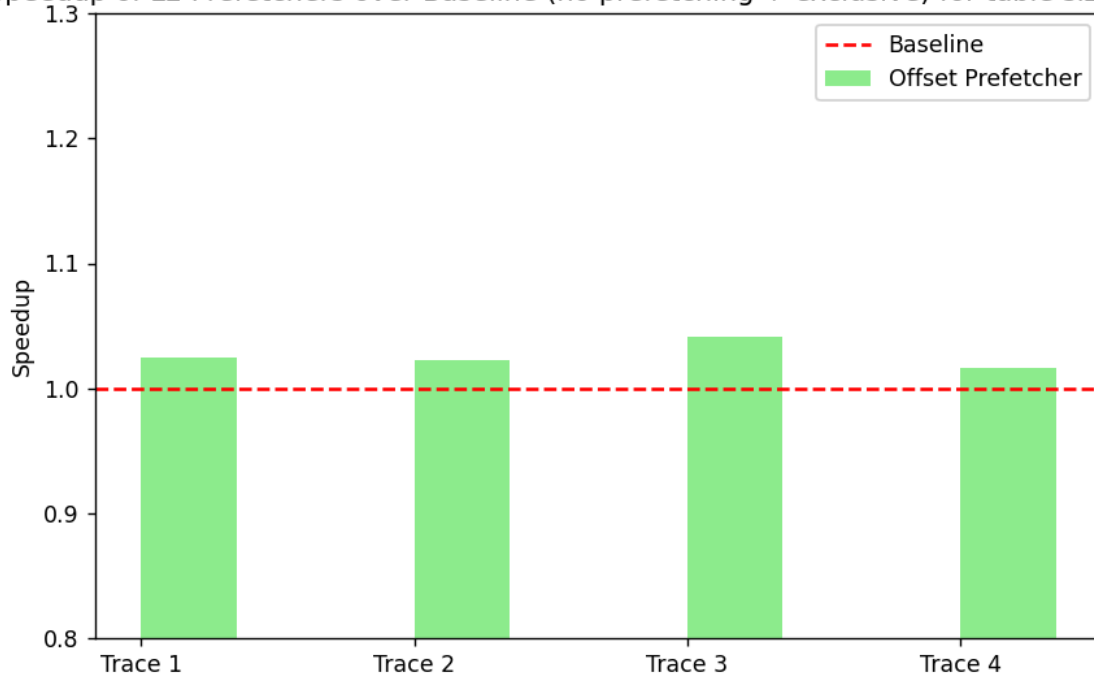
Speedup of L2 Prefetchers over Baseline (no prefetching + exclusive) for table size of 32



Speedup of L2 Prefetchers over Baseline (no prefetching + exclusive) for table size of 64



Speedup of L2 Prefetchers over Baseline (no prefetching + exclusive) for table size of 128



TABLES:

Baseline: no prefetching with non-inculsive cache			
TRACE	IPC	MPKI	Speed UP
trace1	0.529026	40.3453	1
trace2	0.462268	47.6893	1
trace3	0.834309	22.197	1
trace4	0.462276	47.562	1

Baseline: no prefetching with exculsive cache			
TRACE	IPC	MPKI	Speed UP
trace1	0.527951	96.2975	1
trace2	0.458571	113.818	1
trace3	0.83402	52.4443	1
trace4	0.46036	113.422	1

Offset prefetching with table size 32 and exclusive cache				
TRACE	IPC	MPKI	Speed UP non-inculsive	Speed UP exclusive
trace1	0.540792	96.3017	1.022240873	1.024322333
trace2	0.468755	113.823	1.014032985	1.02220812
trace3	0.868082	52.4437	1.040480206	1.040840747
trace4	0.467673	113.421	1.011674844	1.015885394

Offset prefetching with table size 64 and exclusive cache				
TRACE	IPC	MPKI	Speed UP non-inculsive	Speed UP exclusive
trace1	0.540792	96.3017	1.022240873	1.024322333
trace2	0.468847	113.823	1.014232004	1.022408744
trace3	0.868082	52.4437	1.040480206	1.040840747
trace4	0.467444	113.425	1.011179469	1.015387957

Offset prefetching with table size 128 and exclusive cache				
TRACE	IPC	MPKI	Speed UP non-inculsive	Speed UP exclusive
trace1	0.540792	96.3017	1.022240873	1.024322333
trace2	0.468755	113.823	1.014032985	1.02220812
trace3	0.868082	52.4437	1.040480206	1.040840747
trace4	0.467673	113.421	1.011674844	1.015885394